

11.1: Cloud Computing



UMD DATA605 - Big Data Systems

11.1: Cloud Computing

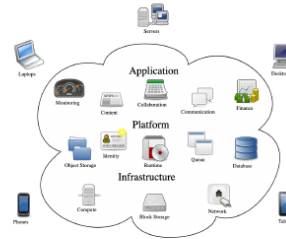
– **Instructor:** Dr. GP Saggese, gsaggese@umd.edu

2 / 10: Cloud Computing

Cloud Computing

- **Computing as "service" instead of "product"**

- Storage and computing in the cloud
- Edge devices (e.g., phones, laptops, tablets) interact with the cloud



- **Advantages of cloud computing**

- Device agnostic: seamless computation across devices
- On demand
- Efficiency / scalability
 - Programming frameworks (e.g., Hadoop, Spark, Dask) enable scalability
- Reliability
- Cost: "pay-as-you-go" for resources
 - Cheaper than building own systems
 - Computing as a commodity (like electricity)

2 / 10

- **Computing as "service" instead of "product"**

- *Storage and computing in the cloud:* This means that instead of owning physical hardware, users can access storage and computing power over the internet. This is flexible and can be scaled up or down based on needs.
- *Edge devices (e.g., phones, laptops, tablets) interact with the cloud:* These devices connect to cloud services to perform tasks, reducing the need for powerful hardware on the device itself.

- **Advantages of cloud computing**

- *Device agnostic:* Users can access cloud services from any device, ensuring a seamless experience across different platforms.
- *On demand:* Resources can be accessed whenever needed, providing flexibility and convenience.
- *Efficiency / scalability:* Cloud platforms use frameworks like Hadoop, Spark, and Dask to handle large-scale data processing efficiently, allowing businesses to grow without worrying about infrastructure limits.
- *Reliability:* Cloud services often have high uptime and data redundancy, ensuring consistent availability.
- *Cost: "pay-as-you-go" for resources:* Users only pay for what they use, making it more economical than maintaining personal hardware. This model treats computing resources like utilities such as electricity.

3 / 10: Buying Infrastructure

Buying Infrastructure

- **To buy or to rent** computers and infrastructure?
 - Trade-off between time and money
 - Very difficult to decide objectively
- **Buying / building infrastructure**
 - Requires time and capital investments (Capex)
 - Especially at the beginning without revenues
 - Smooth cash flow (constant \$/mo) is better than a lumpy one (big one-time purchase)
 - Buy hardware (e.g., computers, storage, network)
 - Estimate current hardware size
 - Difficult to estimate future demands
 - Update obsolete hardware
 - Cost of owning hardware (Opex)
 - Data center, electricity, cooling, handling faults
 - Administering
 - Install, update, maintain software stack

3 / 10

- **Buying Infrastructure**
 - *To buy or to rent* computers and infrastructure?
 - When deciding whether to buy or rent infrastructure, there's a balance between time and money. Buying might save money in the long run but requires a lot of upfront investment. Renting can be more expensive over time but is easier to start with.
 - It's tough to make this decision because it involves predicting future needs and costs, which can be uncertain.
- **Buying / building infrastructure**
 - This involves significant time and capital investments, known as Capex. This is especially challenging at the start when there might not be any revenue to support these costs.
 - A steady cash flow, where you pay a consistent amount each month, is often preferable to a large, one-time expense that can disrupt finances.
 - When you buy hardware like computers, storage, and network equipment, you need to estimate how much you need now. However, predicting future needs can be tricky, as technology and business demands change.
 - Hardware becomes outdated, so you'll need to plan for updates and replacements.
 - Owning hardware comes with ongoing operational expenses (Opex), such as costs for data centers, electricity, cooling, and fixing any issues that arise.
 - Managing the infrastructure also requires effort. This includes installing, updating, and maintaining the software that runs on your hardware.

4 / 10: Renting Infrastructure

Renting Infrastructure

- **Renting infrastructure** (i.e., cloud computing)
 - Pay for what you use
 - Low initial capital investment
 - Ready systems with a click
 - No multi-year resource plan needed
 - Choose machines based on your application and data needs

4 / 10

- **Renting Infrastructure**
 - *Renting infrastructure* refers to using cloud computing services instead of owning physical hardware. This means you can access computing resources over the internet.
 - **Pay for what you use**
 - This model allows you to pay only for the resources you consume, similar to how you pay for utilities like electricity or water. This is beneficial because it reduces the need for a large upfront investment in hardware.
 - **Low initial capital investment**
 - Since you don't need to buy expensive servers or other hardware, you can start projects with minimal financial risk. This is especially useful for startups or small businesses that may not have a lot of capital.
 - **Ready systems with a click**
 - Cloud services provide pre-configured systems that you can deploy quickly. This means you can have a server up and running in minutes, which speeds up development and deployment times.
 - **No multi-year resource plan needed**
 - You don't have to predict your resource needs years in advance. Instead, you can scale your resources up or down based on current demand, making it easier to adapt to changing business needs.
 - **Choose machines based on your application and data needs**
 - Cloud providers offer a variety of machine types optimized for different tasks. You can select the best fit for your specific application, whether you need more processing power, memory, or storage. This flexibility ensures you get the most efficient performance for your workload.

5 / 10: Cloud Computing

Cloud Computing

- **Ideas of cloud computing around for a long time**
 - Mainframes + thin clients (1960s)
 - Personal computers (1980s)
 - Grid computing for supercomputers (1990s)
 - Peer-to-peer architecture (early 2000s)
 - Client-server model (Web 1.0 and Web 2.0)
 - Cloud computing (2010s)
- **Now, it finally works**
- **Why now?**
 - A convergence of key technologies
 - OS virtualization
 - Large data centers
 - Decreasing hardware costs
 - **Big data frameworks**



5 / 10

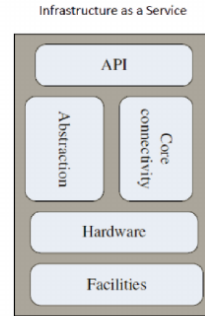
- **Ideas of cloud computing around for a long time**
 - *Mainframes + thin clients (1960s)*: Early computing involved centralized mainframes accessed by simple terminals, laying the groundwork for cloud concepts.
 - *Personal computers (1980s)*: The rise of PCs shifted computing power to individual users but still relied on networked resources.
 - *Grid computing for supercomputers (1990s)*: This era saw the use of distributed computing resources to solve large-scale problems, similar to cloud resource pooling.
 - *Peer-to-peer architecture (early 2000s)*: Enabled decentralized resource sharing, influencing cloud's distributed nature.
 - *Client-server model (Web 1.0 and Web 2.0)*: These models introduced web-based applications, a precursor to cloud services.
 - *Cloud computing (2010s)*: Finally, the cloud became a mainstream solution, offering scalable and flexible computing resources.
- **Now, it finally works**
 - Cloud computing has matured, becoming reliable and efficient for various applications.
- **Why now?**
 - *A convergence of key technologies*:
 - **OS virtualization**: Allows multiple virtual machines on a single physical server, optimizing resource use.
 - **Large data centers**: Provide the infrastructure needed for cloud services, offering vast storage and processing power.
 - **Decreasing hardware costs**: Makes it economically feasible to deploy large-scale cloud solutions.
 - **Big data frameworks**: Enable the processing and analysis of massive datasets, a

key cloud application.

6 / 10: Infrastructure-as-a-Service (IaaS)

Infrastructure-as-a-Service (IaaS)

- Cloud provides low-level resources like virtual machines, storage, and networking
 - Users install and maintain the operating system and applications
- **Examples**
 - AWS EC2
 - Google Compute Engine
 - Microsoft Azure Virtual Machines
 - IBM Cloud Infrastructure
- **Benefits**
 - Flexibility in managing resources
 - Scalability to meet demand
 - Cost-effective, pay for what you use
- **Use Cases**
 - Hosting websites and applications
 - Data analysis and processing
 - Development and testing environments



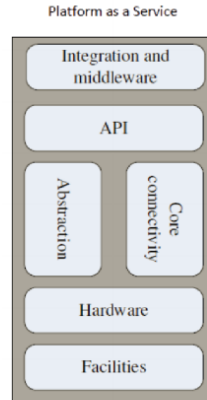
6 / 10

- **Infrastructure-as-a-Service (IaaS)**
 - IaaS provides foundational cloud services, offering essential resources like virtual machines, storage, and networking. Users have control over these resources, allowing them to install and maintain their own operating systems and applications. This setup gives users significant flexibility and control over their computing environment.
- **Examples**
 - *AWS EC2*: Amazon's Elastic Compute Cloud offers scalable computing capacity.
 - *Google Compute Engine*: Provides virtual machines running in Google's data centers.
 - *Microsoft Azure Virtual Machines*: Offers on-demand, scalable computing resources.
 - *IBM Cloud Infrastructure*: Provides a range of cloud computing services.
- **Benefits**
 - *Flexibility*: Users can manage resources according to their needs.
 - *Scalability*: Easily adjust resources to meet changing demands.
 - *Cost-effective*: Pay only for the resources you use, reducing waste.
- **Use Cases**
 - *Hosting websites and applications*: Ideal for businesses needing reliable hosting.
 - *Data analysis and processing*: Supports large-scale data operations.
 - *Development and testing environments*: Provides a flexible space for developers to test applications.

7 / 10: Platform-as-a-Service

Platform-as-a-Service

- **Problem:** assembling your own software stack requires work
 - Install
 - Configure
 - Manage dependencies (e.g., incompatible versions)
- **Solution:** get a pre-built software stack
 - Software stack as a virtualization solution (e.g., VMware or Docker)
 - Pre-installed OS
 - Libraries
 - Application software
- **Business model built around this**
 - E.g.,
 - Google App Engine
 - Managed Hadoop
 - E.g., pre-built images for Hadoop
 - Hortonworks, Cloudera
 - E.g., pre-built distributions for Linux
 - RedHat, Gentoo, CentOS



7 / 10

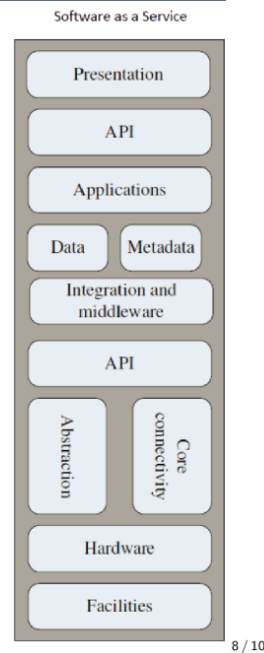
- **Problem: Assembling your own software stack requires work**
 - *Install:* Setting up each component of a software stack can be time-consuming and complex.
 - *Configure:* Ensuring that all components work together smoothly often involves intricate configuration.
 - *Manage dependencies:* Handling incompatible versions of software can lead to significant challenges.
- **Solution: Get a pre-built software stack**
 - *Software stack as a virtualization solution:* Tools like VMware or Docker provide a ready-to-use environment.
 - *Pre-installed OS:* Operating systems are already set up, saving time and effort.
 - *Libraries:* Necessary libraries are included, reducing compatibility issues.
 - *Application software:* Applications are pre-installed, allowing for immediate use.
- **Business model built around this**
 - Examples include:
 - *Google App Engine:* Offers a platform for building and hosting applications.
 - *Managed Hadoop:* Provides a ready-to-use Hadoop environment.
 - Pre-built images for Hadoop:
 - *Hortonworks, Cloudera:* These offer Hadoop distributions with everything set up.
 - Pre-built distributions for Linux:
 - *RedHat, Gentoo, CentOS:* These distributions come with pre-configured software stacks.

The diagram illustrates the components of Platform-as-a-Service (PaaS), highlighting integration, middleware, APIs, and core connectivity, which abstract the complexities of hardware and facilities.

8 / 10: Software-as-a-Service (SaaS)

Software-as-a-Service (SaaS)

- **Cloud provides the application**
 - No need to install on your machine
 - Use directly from the cloud
 - Examples include:
 - *Dropbox*: Access and share files without local storage
 - *Salesforce*: Manage customer relationships online
 - Any app running in a browser: Google Docs or Microsoft 365
- **Benefits**
 - Accessibility from any device with internet connectivity
 - Automatic updates and maintenance by the provider
 - Scalability for growing user needs
 - Cost-effectiveness by reducing physical hardware and software installations



- **Software-as-a-Service (SaaS)**
 - *Cloud provides the application*
 - SaaS applications are hosted in the cloud, meaning you don't need to install them on your local machine. This allows you to access them directly via the internet.
 - Examples include:
 - *Dropbox*: This service lets you store and share files online, eliminating the need for local storage.
 - *Salesforce*: A platform for managing customer relationships, accessible entirely online.
 - Browser-based apps like Google Docs or Microsoft 365, which allow you to create and edit documents directly in your web browser.
- **Benefits**
 - **Accessibility**: You can use SaaS applications from any device with an internet connection, providing great flexibility.
 - **Automatic updates**: The service provider handles updates and maintenance, ensuring you always have the latest features without manual intervention.
 - **Scalability**: SaaS can easily accommodate growing user needs, making it suitable for businesses of all sizes.
 - **Cost-effectiveness**: By reducing the need for physical hardware and software installations, SaaS can lower operational costs.

The accompanying diagram illustrates the layered architecture of SaaS, showing how different components like presentation, APIs, and data interact within the cloud infrastructure.

9 / 10: X-as-a-Service

X-as-a-Service

- After 2010, business model of **X-as-a-Service (XaaS)**
 - Mobility-as-a-Service (e.g., Uber)
 - Gaming-as-a-Service (e.g., Google Stadia)
 - Storage-as-a-Service (e.g., S3, Google Drive)
 - Desktop-as-a-Service (e.g., AWS AppStream)
 - Marketing-as-a-Service
 - Banking-as-a-Service
 - ..

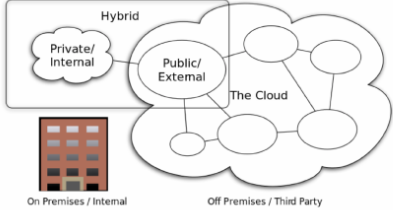
9 / 10

- **X-as-a-Service**
 - **After 2010, business model of X-as-a-Service (XaaS):**
 - **Mobility-as-a-Service (e.g., Uber):** This model allows users to access transportation services via an app, providing convenience and flexibility without owning a vehicle. Uber is a prime example, offering rides on demand.
 - **Gaming-as-a-Service (e.g., Google Stadia):** This service lets users play video games over the internet without needing a gaming console or high-end PC. Google Stadia streams games directly to devices, making gaming more accessible.
 - **Storage-as-a-Service (e.g., S3, Google Drive):** This model provides cloud storage solutions, allowing users to store and access data online. Services like Amazon S3 and Google Drive offer scalable storage options for both individuals and businesses.
 - **Desktop-as-a-Service (e.g., AWS AppStream):** This service delivers virtual desktops to users over the internet, enabling access to a desktop environment from any device. AWS AppStream is an example that supports remote work and flexibility.
 - **Marketing-as-a-Service:** This model offers marketing solutions and strategies as a service, helping businesses reach their target audience without needing an in-house marketing team.
 - **Banking-as-a-Service:** This service provides banking infrastructure and capabilities to businesses, allowing them to offer financial services without being a traditional bank.
 - **..:** The “X” in XaaS can represent virtually any service, highlighting the flexibility and broad applicability of this business model across various industries.

10 / 10: Cloud Deployment Models

Cloud Deployment Models

- **Private**
 - Internal cloud hosted on organizational premises
 - E.g., company's data center running virtualized services
- **Public**
 - External cloud hosted by third-party providers
 - E.g., AWS, Azure, GCP offering compute and storage to clients
- **Hybrid cloud**
 - Combines private and public environments to optimize cost, security, and scalability
 - Sensitive workloads stay internal
 - Scalable tasks move to the public cloud
- **On-premises resources**
 - E.g., corporate servers in a building
- **Off-premises resources**
 - E.g., cloud provider's distributed data centers



The diagram illustrates cloud deployment models. It shows a 'Hybrid' model connecting a 'Private/Internal' cloud (represented by a cloud icon) and a 'Public/External' cloud (represented by a cloud icon). These are connected to 'The Cloud' (represented by a cloud icon). Below the clouds, there are two server racks: one labeled 'On Premises / Internal' and another labeled 'Off Premises / Third Party'.

- **Private Cloud**
 - *Internal cloud hosted on organizational premises:* This means the cloud infrastructure is located within the organization itself, providing more control over data and security.
 - *Example:* A company's data center that runs virtualized services, allowing for customized solutions tailored to specific business needs.
- **Public Cloud**
 - *External cloud hosted by third-party providers:* These are managed by external companies, offering services over the internet.
 - *Example:* Providers like AWS, Azure, and GCP offer scalable compute and storage solutions, making them accessible and cost-effective for businesses.
- **Hybrid Cloud**
 - *Combines private and public environments:* This model leverages the benefits of both private and public clouds to optimize costs, enhance security, and improve scalability.
 - *Sensitive workloads stay internal:* Critical data and applications remain within the private cloud for security.
 - *Scalable tasks move to the public cloud:* Non-sensitive tasks can be shifted to the public cloud to take advantage of its scalability.
- **On-premises Resources**
 - *Example:* Corporate servers located within a company's building, providing direct control over hardware and data.
- **Off-premises Resources**
 - *Example:* Cloud provider's distributed data centers, which offer flexibility and scalability without the need for physical infrastructure management.